Assignment Report

Labyrinth Game

Name: Nooshti Devi Beelur

Student ID: 23448531

Unit code: COMP1000

Table of Contents

Contents

Overview 3

User Guide 4

Discussion 5

Makefile Diagram 11

Conclusion 11

Figure 1 : Game Interface 3

Figure 2: Makefile diagram 11

Overview

The key objects of the main interface of the game are:

- Player (P)
- Goal (G)
- Treasure (T)
- Enemy (<, >, v, ^)
- Map Border (*)
- Wall (‘ ‘)

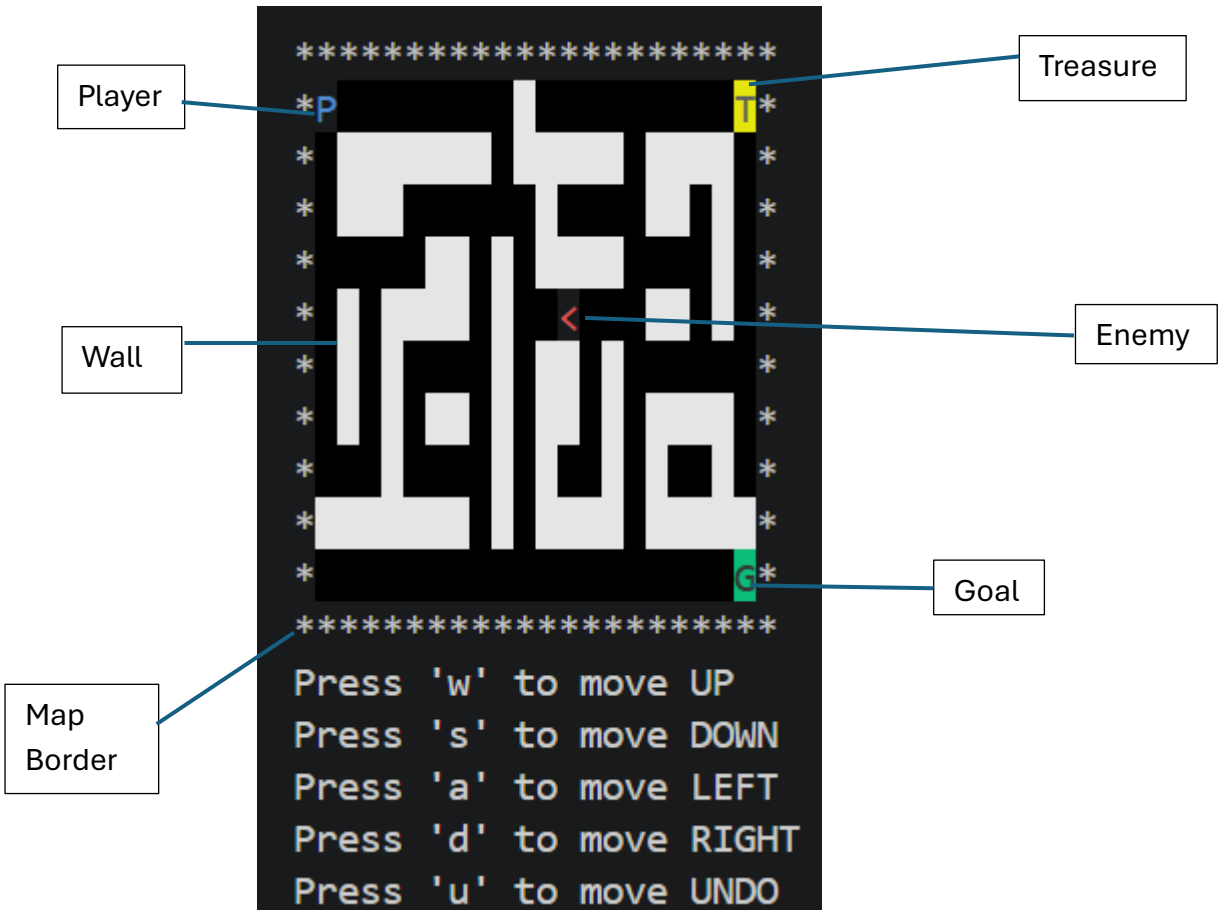


Figure 1 : Game Interface

The game uses a dynamically allocated 2D char array to store extracted information from a text file ‘map.txt’. In order to win the game, the user needs to make the Player loot the Treasure and then reach at the position of Goal without the Enemy catching it. If the Enemy catches the Player, then the user loses the game.

User Guide

1) Extract the files of the zip file

   unzip Veerajay_Gooljar_23448531_Nooshti_Devi_Beelur_Assignment.zip

2) Change to assignment directory from current directory inside a linux terminal

   cd assignment_directory

3) Display all the files inside the assignment directory

   ls

4) Compile the 'Labyrinth' program

   make

5) Run the program

   ./labyrinth map.txt

Gameplay:

- Press 'w' to move P one step up
- Press 's' to move P one step down
- Press 'a' to move P one step left
- Press 'd' to move P one step right
- Press 'u' to undo Player and Enemy movement. It also can undo the looting of Treasure.

Winning condition:

P loots Treasure and reaches Goal without being caught by the Enemy.

Losing condition:

Enemy catches P before the latter reaches Goal.

Note:

- The program is designed to accept lower case letters only so make sure the caps lock is not on while the game is running.
- Any key other than ‘w’, ‘s’, ‘a’, ‘d’, and ‘u’ being pressed will not result in any movement of the Player.
- The command below can be used to verify for memory leaks:
  valgrind ./labyrinth map.txt

Discussion

The assignment folder comprises of a text file ‘map.txt’, a makefile, 6 C files and 5 header files.

Source code files

player.c:

Function	Explanation
void find_position(char **map, int rows, int cols, Position* player, Enemy* enemy, Position* goal_pos)	Finds the position of player, enemy and goal.
int move_player(char **map, int rows, int cols, Position* player, Enemy* enemy, char* key, int* treasure, int* goal, int* got_player)	Contains the algorithm to move the player.
int check_win(int* treasure, int* goal)	Contains code to verify if user has won
int check_lose(int* got_player)	Contains code to verify if user has lost

enemy.c:

Function	Explanation
int is_free(char **map, int rows, int cols, int row, int col)	Verifies if next movement of enemy leads to collide with wall, treasure and goal

char get_direction_enemy(char facing, char direction)	Takes the current enemy direction symbol and future direction to determine the new enemy direction symbol
int can_fwd(char** map, int rows, int cols, Enemy* enemy)	Checks if enemy can move forward
int can_lft(char** map, int rows, int cols, Enemy* enemy)	Checks if enemy can move left
int can_rgt(char** map, int rows, int cols, Enemy* enemy)	Checks if enemy can move right
void move_1_step(char next_dir, Enemy* enemy)	Provides future position to Move enemy by one step
void move_enemy(char **map, int rows, int cols, Enemy* enemy, int *treasure, int* got_player)	Moves enemy by one step after verifying in which direction the enemy will go

map.c:

Function	Explanation
char** load_map(char* filename, int* rows, int* cols)	Reads and processes the map to be stored
void print_map(char** map, int rows, int cols, int *treasure)	Prints the map
char ** copy_map(char** org, int rows, int cols)	Copies the original map shown on screen to be later stored in an undo linked list using a struct
void free_map(char** map, int rows)	Frees all memory allocated to pointer to pointer map
void free_copy(char** copy, int rows)	Frees all memory allocated to pointer to pointer copy map

main.c:

Function	Explanation

int main(int argc, char** argv)	Has all the driver code to run the program

undo.c:

Function	Explanation
void create_undo_list(Undo_list *u_list)	Creates a linked list to store the snapshots of game state
void insert_snapshot_first(Undo_list *u_list, current_state *state)	Inserts snapshots where head node points to
void* pop_snapshot_first(Undo_list *u_list)	Returns and removes the snapshot in first position
void save_snapshot(Undo_list *u_list, char** map, int rows, int cols, int* treasure, Position* player, Enemy* enemy)	Saves snapshots of game state
void undo_game(Undo_list *u_list, char*** map, int rows, int cols, int* treasure, Position* player, Enemy* enemy)	Shows the last snapshot of game state
void free_undo_list(UndoNode *head, int rows)	Frees all memory allocated to undo linked list

terminal.c:

Function	Explanation
void setForeground(char * color)	Used to set the color of Player, enemy, Goal and Treasure
void setBackground(char * color)	Used to set the background color of Player, enemy, Goal and Treasure
void newSleep(float sec)	Delays display by desired number of seconds
void disableBuffer()	Unables the display of chars entered by user

void enableBuffer()	Allows the display of chars entered by user
void initRandom()	Called once at beginning of program before using randomUCP()
int randomUCP(int low, int high)	Generates a random between low and high included

Header files

The functions inside of the source code files have been declared in the following files.

1. *player.h*

```
typedef struct {    /*stores position of given char*/
   int row;
   int col;
} Position;

typedef struct {    /*stores position and direction faced char of enemy*/
   Position pos;
   char dir_facing; /*< > ^ V*/
} Enemy;

void find_position(char **map, int rows, int cols, Position* player, Enemy*;
enemy, Position* goal_pos);
int move_player(char **map, int rows, int cols, Position* player, Enemy* enemy,
char* key, int* treasure, int* goal, int* got_player);

int check_win(int* treasure, int* goal);

int check_lose(int* got_player);
```

2. *enemy.h*

```
int is_free(char **map, int rows, int cols, int row, int col);
char get_direction_enemy(char facing, char direction);
```

```
int can_fwd(char** map, int rows, int cols, Enemy* enemy);

int can_lft(char** map, int rows, int cols, Enemy* enemy);

int can_rgt(char** map, int rows, int cols, Enemy* enemy);

void move_1_step(char next_dir, Enemy* enemy);

void move_enemy(char **map, int rows, int cols, Enemy* enemy, int *treasure,
int* got_player);
```

3. *map.h*

```
char** load_map(char* filename, int* rows, int* cols);

void print_map(char** map, int rows, int cols, int *treasure);

char ** copy_map(char** org, int rows, int cols);
void free_map(char** map, int rows);
void free_copy(char** copy, int rows);
```

4. *undo.h*

```
typedef struct{ /*captures full snapshot of current game state*/
   char **map;
   Position player;
   int has_treasure;
   Enemy en;
}  current_state;

typedef struct UndoNode {
   current_state *data
   struct UndoNode *next;
}  UndoNode;

typedef struct Undo_list {
   UndoNode *head;
```

```
} Undo_list;

void create_undo_list(Undo_list *u_list);
void insert_snapshot_first(Undo_list *u_list, current_state *state);
void* pop_snapshot_first(Undo_list *u_list);
void save_snapshot(Undo_list *u_list, char** map, int rows, int cols, int* treasure,
Position* player, Enemy* enemy);
void undo_game(Undo_list *u_list, char*** map, int rows, int cols, int* treasure,
Position* player, Enemy* enemy);
void free_undo_list(UndoNode *head, int rows);
```

5. *terminal.h*

```
void setForeground(char * color);
void setBackground(char * color);
void newSleep(float sec);

void disableBuffer();
void enableBuffer();
void initRandom();
int randomUCP(int low, int high);
```

Makefile Diagram

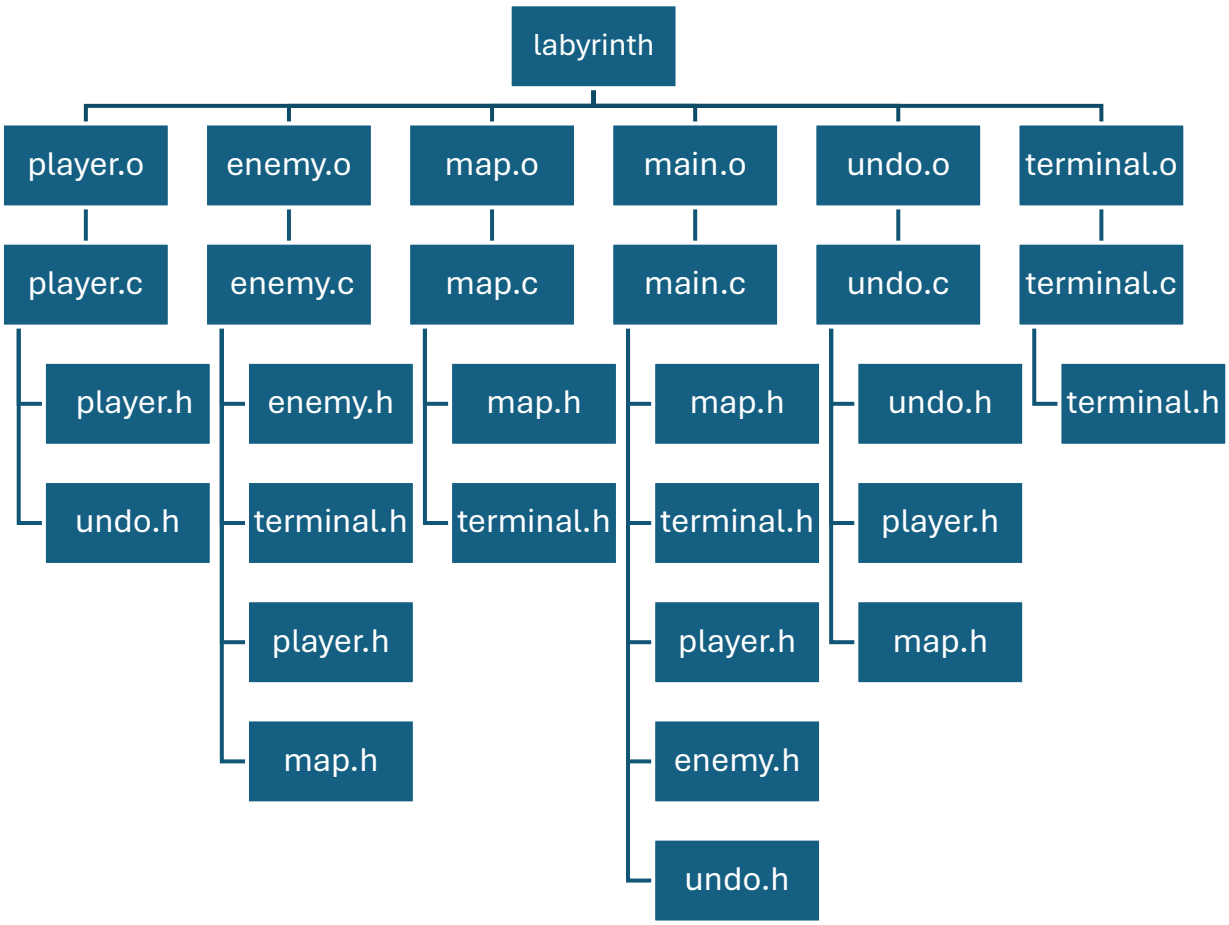


Figure 2: Makefile diagram

Conclusion

I implemented the player and enemy movement, treasure looting and goal detection using the concepts on the lecture slides and the codes provided as additional support for the assignment. Key programming concepts such as 2D arrays, dynamic memory allocation, linked lists, structs and pointers were essential in completing the program.

The above concepts were especially helpful in detecting collision of player with enemy, checking of map borders and to verify win/lose conditions.